

Scene Graph Node TL;DR

A scene graph is a tree structure where each node has a specific rendering responsibility.

Typical rendering flow:

```
CameraNode
  ↓
TransformNode
  ↓
PresentationNode / ShaderNode
  ↓
GeometryNode
```

Core Scene Graph Nodes

Node	Responsibility
SceneNode	Base tree structure
TransformNode	Position/orientation/scale
PresentationNode	Appearance/material settings
ShaderNode	GPU shader control
GeometryNode	Actual rendering/drawing
CameraNode	View/projection setup

SceneNode

SceneNode is the base class for everything in the scene graph.

It provides:

- parent/child relationships
- traversal
- draw()
- update()

Every other node inherits from this class.

TransformNode

TransformNode controls WHERE objects appear.

It applies transformations such as:

- translation
- rotation
- scaling

Typical functions:

```
translate(x, y, z)
rotate_x(deg)
rotate_y(deg)
rotate_z(deg)
scale(x, y, z)
```

It usually does NOT draw geometry itself.

Instead, it modifies the transformation matrix before child nodes draw.

Example:

```
TransformNode
  translate(5,0,0)
    ↓
  GeometryNode(cube)
```

Result:

- the cube appears shifted right by 5 units

Mental model:

“Everything beneath me is transformed.”

PresentationNode

PresentationNode controls HOW objects look.

Typical responsibilities:

- color
- material settings
- lighting values
- shader uniforms
- render state

It usually prepares visual settings before child nodes draw.

Example:

```
PresentationNode
  set color = red
    ↓
  GeometryNode(cube)
```

Result:

- the cube draws red

Mental model:

“I style the object.”

ShaderNode

ShaderNode controls GPU shader programs.

It manages:

- vertex shaders
- fragment shaders
- shader programs

Responsibilities include:

- loading shaders
- compiling shaders
- linking programs
- activating shaders
- locating uniforms and attributes

Important detail:

```
virtual bool get_locations() = 0;
```

This makes **ShaderNode** an abstract base class.

Meaning:

- you cannot instantiate **ShaderNode** directly
- you must derive specialized shader classes

Example:

```
class PhongShaderNode : public ShaderNode
```

Mental model:

“I control how the GPU processes rendering.”

GeometryNode

GeometryNode is where actual rendering happens.

Responsibilities:

- meshes
- vertices
- triangles
- vertex buffers
- draw calls

Typical OpenGL draw calls:

```
glDrawArrays(...)  
glDrawElements(...)
```

Usually a leaf node in the scene graph.

By the time rendering reaches this node:

- transforms are already applied
- materials are already configured
- shaders are already active

Mental model:

“I draw the actual object.”

CameraNode

CameraNode controls how the scene is viewed.

Projection types:

```
enum class ProjectionType  
{  
    PERSPECTIVE,  
    ORTHOGRAPHIC  
};
```

Responsibilities:

- view matrices
- projection matrices
- camera setup

Mental model:

“I define the viewer’s perspective.”

Overall Rendering Flow

Typical traversal:

```
CameraNode  
    setup view/projection
```

```
TransformNode  
    move object
```

PresentationNode
set appearance

ShaderNode
activate shader

GeometryNode
draw mesh

Simple Mental Analogy

Node	Analogy
CameraNode	camera lens
TransformNode	moving the object
PresentationNode	painting/styling the object
ShaderNode	rendering technique/lighting
GeometryNode	actual object mesh

Biggest Conceptual Takeaway

Scene graphs separate responsibilities cleanly:

- WHERE → TransformNode
- HOW IT LOOKS → PresentationNode
- HOW GPU PROCESSES IT → ShaderNode
- WHAT IT IS → GeometryNode
- HOW WE VIEW IT → CameraNode

That separation is the main power of scene graph design.

Scene Graph Node Relationships

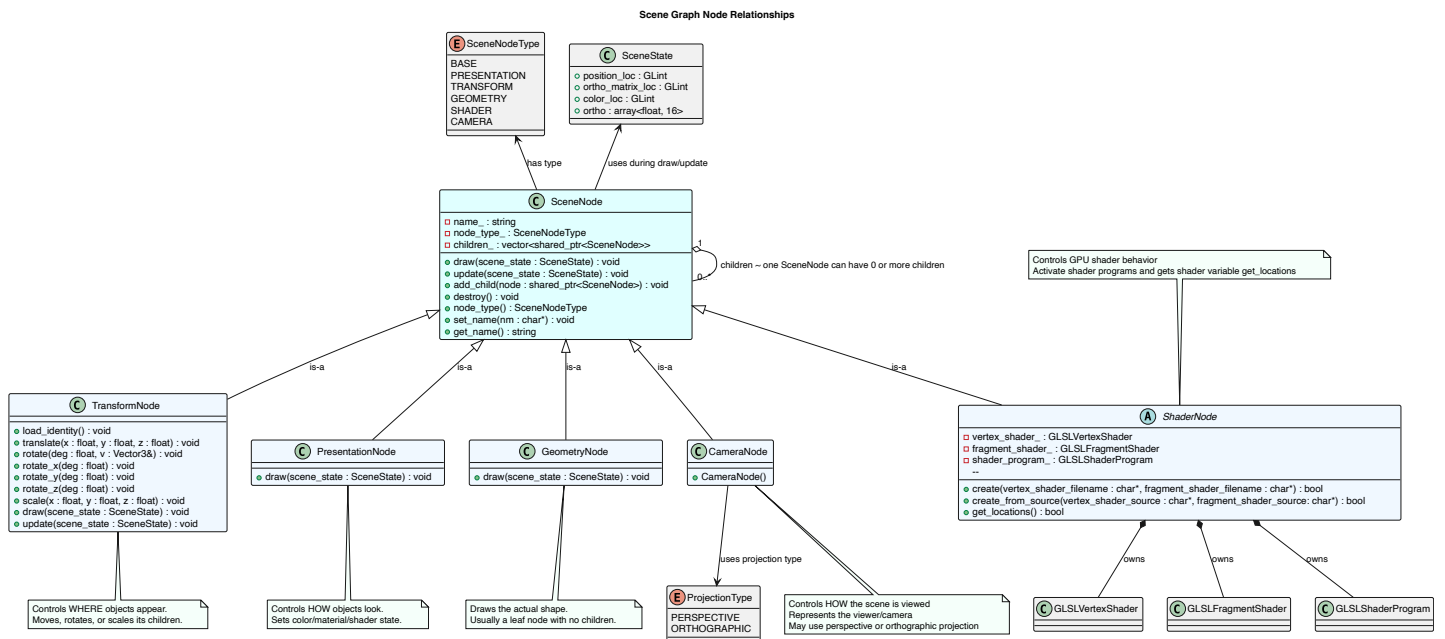


Figure 1: Scene Graph Node Relationships

Generating PDF

```
pandoc readMe.md \  
  --include-in-header=header.tex \  
  -o readMe.pdf
```